

Technology Stack - HouseRent MERN
Date: 18 June 2026
Project Name: House Rent
Maximum Marks: 4 Marks

TECHNOLOGY STACK ANALYSIS

ARCHITECTURE: 3-Tier Web Application

[PRESENTATION] React Frontend

|

REST API

|

[APPLICATION] Express Backend

|

Database Queries

|

[DATA] MongoDB Database

FRONTEND TECHNOLOGIES:

Framework: React 18 (component-based UI)
Build Tool: Vite 5.4.1 (fast bundling)
Styling: Bootstrap 5.3.2 (responsive design)
Advanced UI: Material UI 5.16.4 (components)
HTTP Client: Axios 1.6.5 (API requests)
Routing: React Router v6 (navigation)

Why Frontend Stack:

React: Component reusability, virtual DOM, large community
Vite: Lightning-fast HMR, optimized builds
Bootstrap: Mobile-first grid, pre-styled components
Material UI: Professional design system
Axios: Simple HTTP client with interceptors

BACKEND TECHNOLOGIES:

Runtime: Node.js (JavaScript on server)
Framework: Express.js 4.18.2 (web framework)
Authentication: JWT 9.0.1 (stateless auth)
Password: bcryptjs 2.4.3 (secure hashing)
CORS: cors 2.8.5 (cross-origin requests)
Environment: dotenv 16.3.1 (config management)
Dates: Moment.js 2.29.4 (date handling)

Why Backend Stack:

Node.js: Event-driven, non-blocking I/O
Express: Lightweight, flexible, perfect for APIs
JWT: Scalable stateless authentication
bcryptjs: Industry-standard password hashing

DATABASE TECHNOLOGIES:

Database: MongoDB (NoSQL, flexible schema)
Hosting: MongoDB Atlas (cloud, managed)
ODM: Mongoose 7.5.2 (schema validation)

Why MongoDB:

Flexible JSON-like documents
Scales horizontally with sharding
Perfect for rental data with relationships
Atlas free tier for development

COLLECTIONS:

- Users (name, email, role, approved status)
- Properties (owner, title, location, rent, amenities)

- Bookings (property, tenant, dates, status)
- BookingHistory (user search tracking)

DEVELOPMENT TOOLS:

Package Manager: npm (dependency management)

Auto-restart: nodemon (dev convenience)

Version Control: Git

Editor: VS Code

API Testing: Postman

SECURITY ARCHITECTURE:

- JWT for stateless authentication
- bcrypt for password hashing
- CORS for origin validation
- Role-based middleware for access control
- Input validation on all endpoints
- HTTPS in production

INTEGRATION FLOW:

1. React component sends request via Axios
2. Express endpoint receives and validates JWT
3. Mongoose queries MongoDB for data
4. Results returned to backend
5. Backend sends JSON response to frontend
6. React updates state and re-renders UI

DEPLOYMENT:

Frontend: Built files deployed to CDN/hosting

Backend: Node.js hosting service

Database: MongoDB Atlas (managed)

STACK BENEFITS:

Single language (JavaScript) across stack

Fast development with React

Scalable with Node.js and MongoDB

Open-source (no licensing costs)

Large community support

Easy to maintain and extend